

Event Hook Engine

Technical Deep Dive

Giacomo Checchi

April 30, 2025

1 Introduction

The **Event Hook Engine** is a PowerShell-based monitoring script that observes a selection of critical system-level events in real-time. It is built around the idea of registering asynchronous listeners that emit notifications and logs when specific behaviors occur on the user's machine. The tool is modular, persistent, and lightweight.

2 Launcher Script: `start_engine.bat`

This script bypasses execution policy restrictions and launches the main PowerShell script:

```
@echo off
powershell.exe -ExecutionPolicy Bypass -File "event_hook_engine.ps1"
pause
```

3 Core Modules and Initialization

The engine imports:

- **ScheduledTasks** (for future extensibility)
- **BurntToast**: a PowerShell module used to display native toast notifications

A custom function `Send-Notification($msg)` handles:

1. Formatting messages with a timestamp
2. Displaying toasts via `New-BurntToastNotification`
3. Logging to `$env:USERPROFILE\Desktop\engine_log.txt`

4 Hook #1: USB Insertion/Removal

Mechanism: Uses WMI class `Win32_VolumeChangeEvent` to detect USB plug/unplug events.

- EventType 2 = Device removed
- EventType 3 = Device inserted

Volume detection logic:

- Script retries up to 5 times (0.5s intervals) to resolve the drive letter and filesystem label.
- If detection fails, it still sends a fallback notification.

5 Hook #2: Application Launch / Termination

Mechanism: Uses a .NET timer (`System.Timers.Timer`) with a 4-second interval.

- Captures processes with `MainWindowTitle` `""` (only visible apps)
- Maintains a hashtable `$global:knownProcs` of running app IDs and names
- Compares the current process list against the known one:
 - New process detected → launch notification
 - Missing process ID → close notification

Note: This avoids flooding from background services or headless processes.

6 Hook #3: File System Changes

Mechanism: Uses `System.IO.FileSystemWatcher`

- Monitors the following folders:
 - Desktop
 - Documents
 - Downloads
- Listens recursively with `IncludeSubdirectories = true`
- Reacts to:
 - File creation
 - File modification
 - File deletion
- Filters out events related to its own log file (`engine.log.txt`)

7 Hook #4: CPU Overload Monitoring

(Placeholder if implemented later.)

8 Design Patterns and Considerations

8.1 Asynchronous Event Subscription

All hooks use non-blocking ‘Register-*Event’ mechanisms:

- `Register-WmiEvent` for USB
- `Register-ObjectEvent` for timers and `FileSystemWatcher`

8.2 Thread Safety

Global variables such as `knownProcs` are accessed atomically inside the timer scope, avoiding race conditions due to single-threaded execution in PowerShell.

8.3 Persistence and Resource Use

The engine remains resident and responsive with low CPU cost due to:

- Efficient polling intervals (e.g., 4s for processes)
- Event-driven file system monitoring
- No synchronous waits unless necessary (USB retry logic)

9 Logs and Outputs

Every event produces:

- A toast notification with timestamp and details
- A line in the log file:
 - Located at: `Desktop\engine_log.txt`
 - Format: `[YYYY-MM-DD HH:MM:SS] - Message`
- Optional console output (debug only)

10 Security Considerations

- `-ExecutionPolicy Bypass` may be flagged by some security policies
- The script does not elevate privileges
- Toasts are local and non-intrusive
- Log file is append-only (no overwrite or deletion)

11 Conclusion

The Event Hook Engine is a well-structured and scriptable monitoring solution leveraging native PowerShell and .NET interoperability. It is ideal for tracking real-world usage patterns, user actions, or system behaviors in a lightweight and non-intrusive fashion.